

Настройка импорта

Настройка импорта в прикладном модуле БЛ

Подключить модули

- из репозитория [engine](#):
 - [FileParser](#) - предоставляет Python-библиотеку fileparser, которая предназначена для разбора файлов разных форматов - Excel и DBF; различных типов списков — плоских и иерархических.
 - [FileParserExcel](#) - предоставляет парсеры для разбора Excel-файлов.
 - [FileParserDBF](#) - предоставляет функционал для разбора DBF-файлов.
 - [openpyxl](#);
- из [SBIS SDK](#):
 - Excel - требуется для модуля FileParserExcel;
 - DefusedXml, Image-Py, Image - требуются для модуля openpyxl;
 - Workflow Coordinator Client - требуется для использования API сервиса LRS;
 - Workflow Coordinator Client Py - требуется для использования API сервиса LRS из кода Python.

Для размещения загружаемых файлов можно использовать любой сервис хранения файлов - [Сервис временных файлов \(file-transfer\)](#) или СБИС.Диск. Также файлы можно хранить в основном сервисе приложения, работая с ними через платформенные [методы для загрузки файлов](#).

Создать метод анализа данных и построения превью

В [Genie](#) в прикладном модуле БЛ создайте [метод](#), который получает файл, разбирает часть его строк и возвращает результат разбора в формате JSON в клиентскую часть приложения.

Для анализа файла используйте платформенную Python-библиотеку [fileparser](#), предоставляемую вместе с сервисным модулем FileParser.

В библиотеке описан класс FileParser с методом [create_preview\(\)](#), который по умолчанию разбирает первые 20 строк файла и возвращает результат в формате JSON.

```
def PreparePreview (BLOBID):  
    # Импорт библиотеки.  
    import fileparser  
  
    # Загрузка файла.  
    rpc_file = sbis.FileTransfer.Download(file_id, 'spp_admin_input')  
  
    # Создание парсера.  
    parser = fileparser.FileParser()  
  
    # Разбор первых 20 строк файла.  
    preview_data = parser.create_preview(rpc_file)  
  
    return preview_data
```

Ответ метода нужно использовать для открытия панели ["Настройка импорта"](#).

Создать метод запуска длительной операции импорта

Разбор файлов с большим числом строк целесообразно выполнять в фоновом режиме, чтобы не блокировать дальнейшую работу пользователя с интерфейсом веб-приложения. Для этого в сервисном фреймворке

существует [Сервис длительных операций](#).

[Подробнее о создании длительных операций](#)

```
def CreateLongOperation(LRSToken, BLOBID, Params):
    # Импорт библиотеки.
    import workflow2

    # Получаем билдер для описанного в dws-файле сценария
    builder = workflow2.WorkflowBuilder('ImportRegistry')

    # Конфигурация сценария.
    builder.SetDescriptionArgs("некий реестр")

    # Задаем настройки длительной операции.
    settings = workflow2.LongRequestSettings()
    settings.SetCustomData({"id": str(uuid.uuid4()), "type":
"ImportRegistry"})
    settings.SetUiConnectionToken(LRSToken)
    builder.SetLongRequestSettings(settings)

    # Создаём сценарий.
    scenario = builder.Build()

    # Создание задач.
    import_task = scenario.CreateTask('ImportData.Process')
    import_task.service = sbis.Configuration.SystemName()
    import_task.user = sbis.Session.UserID()

    # Задаем Runtime аргументы метода БЛ.
    import_task.SetMethodArgs(BLOBID, Params)

    # Добавляем задачу в сценарий.
    scenario.AddTask(import_task)

    # Иницилируем выполнение сценария на Workflow-сервисе.

workflow2.Sender().AddWorkflow(scenario).Commit(sbis.PublicationPolicy.ppIMME
DIATELY)
```

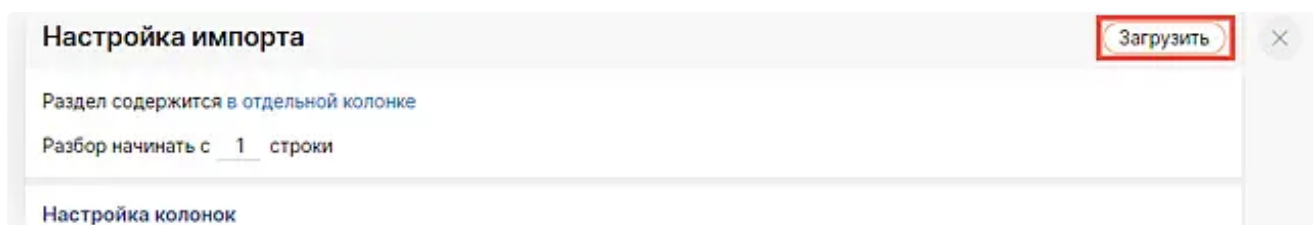
Создать метод импорта

В [Genie](#) в прикладном модуле БЛ создайте [метод](#), который выполняет разбор файла через `FileParser.process` и получение `RecordSet'a`.

В методе определите, как минимум, один аргумент с типом данных "Hash Table (JSON)". В этот аргумент будут передаваться параметры импорта для разбора файла, которые пользователь устанавливает на панели "Настройка импорта".

Для разбора файла следует также использовать библиотеку `fileparser`. В библиотеке описан класс `FileParser` с методом `process()`, который разбирает все строки файла с учетом полученных пользовательских настроек.

Метод `FileParser.process` вызывается при нажатии кнопки "Загрузить" на панели "Настройка импорта".



```

def Process(BLOBID, Params):

    # Получение RPC файла по его идентификатору.
    file = ReadFile(BLOBID)

    # Импорт библиотеки.
    import fileparser

    # Создание парсера.
    parser = fileparser.FileParser()

    # Полный разбор RPC файла.
    # Ответ метода в формате: [{'name': <имя листа>, 'data': <тип
sbis.RecordSet>}]
    data = parser.process(file, Params)

    # Прикладная логика по сохранению данных в БД.
    ...

    # Т.к. это последняя задача в сценарии длительной операции,
    # то из неё возвращается сообщение в пользовательский интерфейс.
    # Благодаря этому может выводиться плашка с результатами длительных
операций.
    record = Record()
    record.AddString("ResultMessage", "Файл Excel распознан.")

    return record

```

Блочное чтение

Блочное чтение требуется при неизвестном и потенциально большом размере файла.

Для чтения используется FileParser. Однако, если полное чтение документа требует только вызова метода `process`, то для итеративного чтения необходимо:

1. задать источник данных (метод `set_source`);
2. итерироваться по листам документа из шаблона импорта;
3. в цикле вызвать метод чтения блока данных `process_lines(sheet: int, lines: int)`.

Пример кода для выгрузки блоками:

```

def ProcessIterative(BLOBID, Params):

    # Получение файла по его идентификатору.
    file = ReadFile(BLOBID)

    # Импорт библиотеки
    import fileparser

    # Создание парсера
    parser = fileparser.FileParser()

    # Задаем размер блока
    batch_size = 50

    # Задаем для парсера источник данных
    parser.set_source(file, Params)

```

```

# Выполняем чтение отдельного листа
def read_sheet(sheet_index):
    has_more = True
    while has_more:
        # Читаем с листа заданное число строк, получаем объект типа
sbis.RecordSet
        lines_data = parser.process_lines(sheet_index, batch_size)

        # Прикладная логика по сохранению данных в БД.
        ...
        has_more = bool(lines_data)

# Параметры разбора могут требовать чтения указанных листов, либо всех
листов документа
if Params.get('same_sheet_configs', False):
    # При обработке всех листов метод парсера get_sheets_number выдает
число листов
    for sheet_index in range(parser.get_sheets_number()):
        read_sheet(sheet_index)
else:
    # При индивидуальных настройках листов параметр sheets хранит
настройки листа, номер листа передается в подполе index
    for sheet_config in Params['sheets']:
        sheet_index = sheet_config['index']
        read_sheet(sheet_index)

# Т.к. это последняя задача в сценарии длительной операции,
# то из неё возвращается сообщение в пользовательский интерфейс.
# Благодаря этому может выводиться плашка с результатами длительных
операций.
record = Record()
record.AddString("ResultMessage", "Файл Excel распознан.")
return record

```

Провайдер обработчика загрузки

Провайдер обработчика загрузки - это модуль БЛ, который позволяет построчно преобразовывать данные из документа. Примером такого провайдера может быть [провайдер сервиса прикладного кода](#).

Редактор типа служит для выбора обработчика и настройки его поведения. Одним из вариантов редактора типа является редактор прикладного кода.

Чтобы реализовать провайдер обработчика необходима поддержка самого провайдера и редактора типа обработчика для обеспечения выбора обработчика на клиентском интерфейсе.

Настройка поддержки провайдера обработчика

1. Для провайдера обработчика создайте в Genie модуль БЛ. Созданный модуль должен иметь зависимости от модуля [FileParser](#) и прикладного модуля, выполняющего вычисления, например, [провайдер сервиса прикладного кода](#).
2. Реализуйте интерфейс [BaseFileDataHandler](#), например, [AppliedCodeFileDataHandler](#).
3. Зарегистрируйте интерфейс в репозитории провайдеров обработчиков с помощью метода [fileparser.FileDataHandlerRegistry.register\(\)](#).

[Пример регистрации интерфейса](#)

Настройка поддержки редактора типа

1. Создайте клиентскую библиотеку для редактора типа;
2. Укажите ее имя в наследнике [BaseFileDataHandler.ui_editor_module](#).

В этой библиотеке должен быть описан следующий контракт опций:

Имя опции	Тип	Комментарий
name	строка	Имя редактора.
editorTemplateName	строка	Имя библиотеки, которая открывает окно редактора для выбора обработчика, например, WasabyClientACS/editor:Reader.
editorOptions	объект	Предустановленные опции для окна редактора.
keyProperty	строка	Имя поля выходной записи редактора, в котором будет находиться идентификатор функции обработчика.
displayProperty	строка	Имя поля выходной записи редактора, содержащее отображаемое пользователю имя обработчика.
additionalTextProperty	строка	Имя поля выходной записи редактора, содержащее дополнительный комментарий по механике работы обработчика.

Настройка импортаЗагрузить

test1073313783

Раздел содержится в отдельной колонке

Разбор начинать с 1 строки и учитывать символы-разделители иерархии:

Обработчик загрузки сотрудников из Excel x ← **displayProperty**

Обработчик импорта из Файла. Поля объекта Вход совпадают с колонками в файле. Поля объекта Выход совпадают с колонками реестра.

Настройка колонок ← **additionalTextProperty**

Пример.

Редактор прикладного кода позволяет оперировать входом/выходом данных - доступом к данным строки. Поля объекта Вход совпадают с колонками в файле. Поля объекта **Выход** совпадают с колонками реестра.

```
Выход.price = Вход.fld_14 + Вход.fld_15;
```

Параметры загружаемого файла fld_14 и fld_15 - это цена и наценка, соответственно. В реестре итоговая цена будет рассчитана как сумма цены и наценки.

Итеративный импорт

При импорте документов с большим объемом данных процесс разбора документа и его записи в прикладную БД может быть долгим, что может приводить к накоплению очередей и другим нежелательным эффектам. Для решения этой проблемы предусмотрены инструменты для построения и выполнения длительных сценарием итеративного импорта.

Основные этапы итеративного импорта:

1. Формирование длительной операции импорта. Выполняется на основном сервисе. Для упрощения построения стандартных сценариев реализован Python-класс *IterativeImportScenarioBuilder*, достаточно определить наследника с необходимыми именами методов записи данных. В особых случаях построением сценария необходимо реализовать самостоятельно.
2. Разбор Excel-документа. Выполняется сервисом [import-excel](#), считанные данные сохраняются как блоки данных в сессиях сервиса **report-prefetch**.
3. Итеративная запись данных в прикладную БД. Выполняется на основном сервисе. Для упрощения чтения данных сессий **report-prefetch** реализован Python-класс *IterativeImporter*, в наследнике достаточно реализовать свой метод

записи данных (например, вызвать уже существующий метод).

4. Завершение сценария. Формирование результата длительной операции.

Техническая документация по организации итеративного импорта доступна в [Базе знаний](#) в разделе "[Сервис итеративного импорта \(import-excel\)](#)"

Форматы полей и авто-исправление ошибок

Метод чтения Excel-документа `FileParser.process` поддерживает автоматическую проверку формата данных ячеек. Формат ячеек должен быть указан в разделе описания листов `sheets`, для каждого проверяемого столбца.

Описание формата поддерживает ограниченный набор базовых типов данных, разделители блоков и значений, а также константные блоки.

Названия типов данных заключаются в фигурные скобки. Если типизированный блок не обязателен, перед закрывающей фигурной скобкой следует добавить вопросительный знак.

Константные блоки должны присутствовать в каждом значении, они входят в описание формата непосредственно.

В качестве разделителей блоков и значений поддерживаются символы: тире, запятая, точка, точка с запятой, двоеточие, прямой и обратный слэши. Разделители блоков в формате указываются непосредственно и используются для отделения типизированных блоков в формате ячейки. Разделители значений заключаются в фигурные скобки и используются между отдельными значениями, если в ячейке может находиться несколько. Если число значений в ячейке не известно, описание формата должно завершаться опциональным разделителем значений, с вопросительным знаком перед закрывающей фигурной скобкой.

Пример шаблона импорта с заданным форматом ячеек:

```
1  {
2      "add_line_number": true,
3      "sheets": [{
4          "index": 0,
5          "skipped_rows": 1,
6          "columns": [{
7              "field": "order",
8              "index": 0,
9              "format": "{date}-{number}{;?}"
10         }, {
11             "field": "date",
12             "index": 1,
13             "format": "{date}"
14         }, {
15             "field": "price",
16             "index": 2,
17             "format": "{decimal}"
18         }
19     ]
20 }
```

Если приведение данных в требуемый формат возможно, то парсер вернет исправленные данные. Если приведение данных не возможно, например, в поле где должно быть число написана строка, парсер вызовет сериализуемое исключение `FieldFormatError` с идентификатором `315a6b1a-2c92-40b7-8425-88db52577eb5`.

Для типа `decimal`, если он единственный в описании формата, поддержано расширенное исправление ошибок: между цифрами удаляются пробельные символы, исправляется дублирование разделителей.

Формат ячеек является опциональным параметром, если он не передан то парсер должен возвращать данные из файла в неизменном виде.

Описание поддерживаемых форматов

Поддерживаются следующий типы данных:

- `integer` - целое число
- `decimal` - десятичная дробь
- `date` - полная дата, при валидации приводится к ISO-формату
- `date_dmy` - полная дата в формате "`день.месяц.год`"

- year - полный год
- month - номер месяца (начиная с единицы)
- day - день месяца
- hour - час
- minute - минута
- second - секунда

Импорт из Excel в модель данных ФЭД

Импорт ФЭД из Excel-файла решает задачу преобразования документа пользователя в модель по спецификации его формата. Ожидается, что документ точно соответствует спецификации, при несоответствии структуры возвращается ошибка.

Преобразование выполняется методом БЛ, в параметрах передаются RPC-файл и [JSON-спецификация](#) формата.

Импорт из Excel выполняется методом БЛ `FileParser.GetData/2`, с параметрами:

- `File` - RPC файл Excel-документа,
- `Specification` - [JSON-объект спецификации](#) формата.

Возвращается JSON-объект данных в [иерархической модели](#).

Расширенные возможности импорта предоставляет метод БЛ `FileParser.GetData/3`, с параметрами:

- `File` - RPC файл Excel-документа,
- `Specification` - [JSON-объект спецификации](#) формата.
- `Options` - JSON-объект с дополнительными настройками импорта, может содержать поля:
 - `SheetsHeaders` - списки возможных неформатных заголовков по листам документа, с подполями
 - `Sheet` - номер листа, начиная с нуля
 - `Rows` - описания возможных строк
 - `Text` - содержимое строки

Если передана опция `SheetsHeaders`, при чтении соответствующего листа игнорируются все строки, содержащие в первом столбце значение, совпадающее с любым из указанных в списке `Rows`. После получения строки с иным значением обработка продолжается в соответствии со спецификацией.

Возвращается JSON-объект данных в [иерархической модели](#).

Методы содержатся в модуле FileParser, для корректной работы необходим также модуль FileParserExcel и удовлетворение его зависимостей от модулей Excel, DefusedXml и File-transfer Client.

Настройка импорта в прикладном модуле UI

Настройка импорта выполняется в прикладном реестре и реализуется модулем [Import](#), задается список ожидаемых полей и их иерархия. Подробнее о настройке клиентской части в [базе знаний wasaby.Frontend](#).